

A Hardware/Software Co-Optimization of HQC

Using Tightly-Coupled Accelerators on a 32-bit Ibex Core

Seong Chung Seo[✉] and YoungBeom Kim^{*✉}

Abstract—We present Hardware/Software co-optimization of Hamming Quasi-Cyclic (HQC) enabled by tightly coupled accelerators implemented on a 32-bit Ibex RISC-V core. On the hardware side, we propose a unified multiplier capable of efficiently performing carryless multiplication for both polynomial multiplication over $\mathbb{F}_2[X]/(X^n - 1)$ and multiplication over $\mathbb{F}_{2^8}$. We also design a Keccak permutation accelerator to support efficient randomness sampling. On the software side, we identify the optimal combination of Toom-Cook and Karatsuba methods for efficient polynomial multiplication on the Ibex core and enhance its performance by minimizing the number of memory accesses during its execution. With our co-optimization strategies, our HQC implementation achieves a performance improvement of several tens of times over the reference implementation.

Index Terms—PQC, HQC, RISC-V, Ibex-core, Accelerator, ISE

I. INTRODUCTION

Hamming Quasi-Cyclic (HQC) is a recently NIST-selected standard key encapsulation mechanism (KEM) [1]–[3]. Unlike ML-KEM [4], which is based on the Learning With Errors problem and was standardized earlier, HQC is code-based and is expected to complement ML-KEM from a crypto-agility perspective [5]. However, HQC is slower than ML-KEM on typical PCs because it relies heavily on large-degree polynomial multiplication (PM) over $\mathbb{F}_2[X]/(X^n - 1)$ ¹. This gap becomes even larger on embedded devices, as Table I shows that HQC is roughly $\times 47.00$ – 108.96 slower than ML-KEM, hindering HQC deployment in many Internet-of-Things (IoT) applications. This widening disparity stems from the fact that typical PCs provide carryless multiplication instructions whereas embedded processors usually lack such support².

RISC-V is an open instruction set architecture that enables application-specific customization via custom instruction extensions, making it attractive for IoT platforms. Several studies have proposed custom instructions to accelerate PQC workloads on RISC-V [7]–[12]. However, most prior work [7]–[11] has focused on lattice-based schemes, including ML-KEM [4] and ML-DSA [13]. Recently, A. Dolmeta et al. [12] proposed the first tightly coupled accelerators targeting large-degree PM , $\mathbb{F}_{2^8}$ arithmetic, and the Keccak permutation, integrated via the CV-X-IF [14] interface on the CV32E40PX RISC-V core [15]. As follow-up work, they also presented tightly coupled unified accelerators for both HQC and ML-KEM on the same CV32E40PX core [16]. Although their approach

This research was supported by Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education (No. RS-2025-25411243, 100%). Seong Chung Seo and YoungBeom Kim are with the Department of Cyber Security, Kookmin University, Seoul 02707, Republic of Korea (e-mail: seseo, darania@kookmin.ac.kr). (Corresponding author: YoungBeom Kim.)

¹According to [2], HQC is about 4.4 – $11.9\times$ slower than ML-KEM on a typical PC.

²Some RISC-V cores, including Ibex, provide carryless multiplication instructions, but enabling them via advanced synthesis configurations can significantly increase hardware resource usage [6].

TABLE I
RUNNING TIME COMPARISON BETWEEN ML-KEM AND HQC FOR SECURITY LEVEL 1 ON IBEX CORE (COMPILED WITH OPTIMIZATION LEVEL -O3 ON IBEX CONFIGURED WITH MAXPERF. cc: CLOCK CYCLE)

	KeyGen (cc)	Encaps (cc)	Decaps (cc)
HQC1 (Ibex)	47,761,436	95,293,337	14,4408,626
ML-KEM 1 (Ibex)	1,016,171	1,127,706	1,325,329

TABLE II
PERFORMANCE PROFILING OF HQC-128 ON IBEX CORE

Function	KeyGen (cc, portion)	Encaps (cc, portion)	Decaps (cc, portion)
Poly_Mul	46,430,152 97.21%	92,860,298 97.45%	139,290,450 96.46%
Sampling	1,261,041 2.64%	1,779,581 1.87%	2,165,924 1.5%
Encode	-	95,292 0.10%	95,292 0.07%
Decode	-	-	1,289,817 0.89%
Total	47,761,436	95,293,337	144,408,626

achieves $\times 17$ – $\times 21$ speedups in [12] and $\times 21$ – $\times 34$ in [16] over the HQC reference implementation, it considers only the Karatsuba-based PM in the reference software and largely applies the proposed custom instructions in a straightforward manner. Moreover, in their work, the multiplier for PM and the multiplier for $\mathbb{F}_{2^8}$ arithmetic are implemented as separate units, and thus the hardware resource usage is not optimized.

In this paper, we present a highly optimized HQC implementation on a 32-bit Ibex core via *HW/SW* co-optimization. On the *HW* side, we propose (i) a unified carryless multiplier supporting both PM over $\mathbb{F}_2[X]/(X^n - 1)$ and multiplication over $\mathbb{F}_{2^8}$, and (ii) a Keccak permutation accelerator to speed up randomness sampling. On the *SW* side, we apply the optimal combination of Toom-Cook and Karatsuba methods for efficient PM and enhance its performance by minimizing the number of memory accesses during its execution. With only the carryless multiplier, we achieve speedups of $\times 26.80$ – $\times 39.52$ (KeyGen), $\times 29.06$ – $\times 41.88$ (Encaps), and $\times 24.96$ – $\times 39.22$ (Decaps) over the HQC reference, with 686 LUTs and 126 FFs overhead. With both accelerators, speedups increase to $\times 40.58$ – $\times 48.21$, $\times 42.94$ – $\times 51.47$, and $\times 37.56$ – $\times 50.31$, at an additional cost of 4,550 LUTs and 1,757 FFs.

II. BACKGROUND

A. Hamming Quasi-Cyclic (HQC)

HQC is a NIST-selected KEM and its security is based on the hardness of syndrome decoding over linear quasi-cyclic codes [1]. The KEM version of HQC consists of KeyGen, Encaps, and Decaps and it operates over $\mathcal{R} = \mathbb{F}_2[X]/(X^n - 1)$ where $n = 17,669, 35,851$, and $57,637$, for security level 1, 3, and 5, respectively.

1) *Profiling of HQC*: To find performance bottlenecks, we conducted a profiling of the latest official HQC-128 reference from [17] on the Ibex-core. From the table II, we find that PM accounts for the majority of HQC’s execution time. Sampling

$B = (b_1 \| b_0) = (b_3 \| b_2 \| b_1 \| b_0)$. Applying the first layer of 2-way Karatsuba, the 32-bit product C is computed using three 16-bit intermediate products (T_0, T_1, T_2) :

$$C = T_2 2^{32} \oplus (T_2 \oplus T_1 \oplus T_0) 2^{16} \oplus T_0, \quad (1)$$

where $T_0 = a_0 \cdot b_0$, $T_2 = a_1 \cdot b_1$, $T_1 = (a_1 \oplus a_0) \cdot (b_1 \oplus b_0)$.

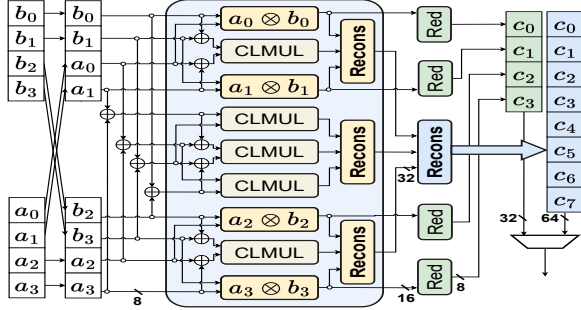


Fig. 2. Proposed Unified 32-bit Carryless Multiplier (Among nine CLMUL units, yellow-colored four are used for four multiplication in $\mathbb{F}_{2^8}$)

Each 16-bit multiplication (T_k) is again decomposed into 8-bit operations using a second layer of Karatsuba. The terms T_0 and T_2 are expanded as:

$$T_0 = (a_1 b_1) 2^{16} \oplus (a_{10} b_{10} \oplus a_1 b_1 \oplus a_0 b_0) 2^8 \oplus (a_0 b_0), \quad (2a)$$

$$T_2 = (a_3 b_3) 2^{16} \oplus (a_{32} b_{32} \oplus a_3 b_3 \oplus a_2 b_2) 2^8 \oplus (a_2 b_2). \quad (2b)$$

Similarly, the middle term T_1 is computed using recursive terms t_0, t_1, t_2 :

$$T_1 = (t_2) 2^{16} \oplus (t_2 \oplus t_1 \oplus t_0) 2^8 \oplus (t_0), \quad (3)$$

where the auxiliary 8-bit terms and products are defined as:

$$t_0 = (a_{20}) \cdot (b_{20}), \quad t_1 = (a_{3210} \cdot b_{3210}), \quad t_2 = a_{31} \cdot b_{31}$$

$$a_{ij} = a_i \oplus a_j, b_{ij} = b_i \oplus b_j, a_{3210} = \bigoplus_{i=0}^3 a_i, b_{3210} = \bigoplus_{i=0}^3 b_i$$

The proposed unified multiplier requires nine 8-bit by 8-bit carryless multiplication units CLMUL; among them, four CLMUL units (blue-colored $a_0 \cdot b_0$ and $a_1 \cdot b_1$ in T_0 , and $a_2 \cdot b_2$ and $a_3 \cdot b_3$ in T_2) are reused to compute four simultaneous multiplications over $\mathbb{F}_{2^8}$. The detailed structure is given in Fig. 2. In the figure, Recons is the Karatsuba reconstruction unit and Red is the reduction unit which reduces the result of multiplication in $\mathbb{F}_{2^8}$. Since the irreducible polynomial used in $\mathbb{F}_{2^8}$ is $f(X) = X^8 + X^4 + X^3 + X^2 + 1$, we optimize its hardware logic by using its special structure. In the case of 64-bit by 64-bit carryless multiplications (e.g., Karat1/2/3), the unified multiplier outputs a 64-bit result. For multiplications over $\mathbb{F}_{2^8}$, the operands a_0 – a_3 and b_0 – b_3 are first loaded into the multiplier via the PACK instruction in Fig. 1. The multiplier then computes the four products $a_0 b_0$, $a_1 b_1$, $a_2 b_2$, and $a_3 b_3$, reduces them using the Red unit, and packs the reduced results into a single 32-bit return value.

3) *Keccak Operation*: Since the Keccak permutation operates on a 1600-bit state, it cannot be kept in 32×32 -bit registers in the Ibex core. Therefore, our accelerator offers dedicated 1600-bit state registers. As shown in

TABLE III
PERFORMANCE IMPROVEMENT WITH PROPOSED ACCELERATORS
(PARENTHESES DENOTE THE EXECUTION TIME AND SPEEDUP WHEN GF_MUL IS EXECUTED FOUR TIMES)

Function	SW (cc)	ISE-based (cc)	Improvements
MULT_64	2,444	31	$\times 78.84$
GF_MUL	179 (716)	24 (24)	$\times 7.46$ ($\times 29.83$)
Keccak-f	25,447	245	$\times 103.87$

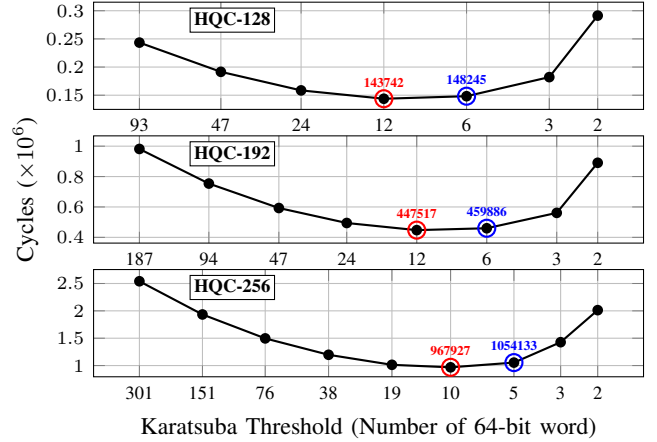


Fig. 3. Computation time as a function of the Karatsuba threshold. The red and blue circles denote the optimal and sub-optimal thresholds, respectively.

Fig. 1, its finite state machine consists of four states (KECCAK_IDLE, KECCAK_LOAD, KECCAK_PERM, and KECCAK_STORE) each corresponding to a dedicated instruction. KECCAK_IDLE resets the internal state. The input is then loaded by issuing KECCAK_LOAD 25 times, where each invocation transfers 64-bit (two 32-bit source registers) into the state registers. The 24-round permutation is executed by invoking KECCAK_PERM 24 times (one round per instruction), and the final 1600-bit state is exported back to the register file via 50 KECCAK_STORE instructions.

Table III compares the (64-bit by 64-bit carryless multiplication (MULT_64), $\mathbb{F}_{2^8}$ multiplication (GF_MUL), and the Keccak permutation) through the proposed accelerators with baseline C implementations and reports substantial speedups. The latency of GF_MUL includes both the 8-bit by 8-bit carryless multiplication and the $\mathbb{F}_{2^8}$ reduction.

B. Software Optimization Techniques

From a SW perspective, we apply a hybrid of TC3 and KA2 to the ref version of HQC³, which originally relied solely on KA for computing PM. Furthermore, we accelerate PM using the proposed carryless multiplication instruction by minimizing memory-access operations.

1) *Finding an Optimal Threshold in Karatsuba*: To optimize PM, we apply TC3 at the top level and then apply recursive KA2 in the subsequent levels. To achieve optimal performance with recursive KA2, it is necessary to identify a threshold at which the recursion should stop and switch to the schoolbook method. For HQC-128/192/256, $n = 17, 669$,

³Actually, we evaluated the performance of the HQC implementation after applying the TC3 + KA2 and the TC4 + KA2 combinations, respectively, and confirmed that the former achieves better performance on the Ibex core.

35,851, and 57,637, corresponding to 277, 561, and 901 64-bit words. With $TC3$, this leads to five KA applications on sub-polynomials of 93, 187, and 301 words, respectively. We determine the optimal KA threshold by measuring execution time while sweeping the threshold (using the ISE-based 64-bit by 64-bit carryless multiplication with our proposed accelerator). Fig. 3 shows how the execution time varies as the threshold value is changed. Although using thresholds of 6, 6, and 5 in KA for HQC-128, HQC-192, and HQC-256 is slightly slower than using 12, 12, and 10, we adopt these values as the thresholds in our implementation. The rationale is that they enable a memory-access-optimized design, allowing us to minimize load/store operations by keeping operands and intermediate values in registers as much as possible.

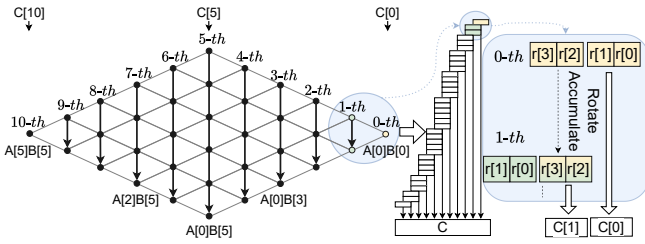


Fig. 4. Proposed PS-based 6-word Schoolbook Multiplication

2) *Memory-access Minimization Strategy for PM*: In the previous section, we selected KA thresholds of 6, 6, and 5 for HQC-128, HQC-192, and HQC-256, respectively, although they are not optimal. These thresholds, however, enable a highly efficient schoolbook multiplication by minimizing memory accesses, yielding better overall performance. For 5- or 6-word (64-bit) polynomial products, we propose a product-scanning (PS)-based carryless multiplication method (Fig. 4) [21]. Both input polynomials are first fully loaded into the Ibex general-purpose registers, after which 64-bit by 64-bit carryless multiplications are executed column-wise from left to right, and within each column from top to bottom.

To accumulate partial products efficiently, we propose a rotating-accumulator scheme that requires only four registers, rather than the n accumulators typically used in naive implementations. After completing the i -th column, the lower two accumulators are written to $C[i]$ and then rotated to serve as the upper accumulators for the $(i+1)$ -th column (Right side in Fig. 4 shows the example for first two columns); this procedure repeats until the final column is processed. Overall, our method fits within 30 registers (12 for the multiplicand, 12 for the multiplier, 4 for the accumulators, and 2 for temporaries), so memory accesses occur only when loading operands and storing the per-column outputs. Consequently, the number of memory-access operations is reduced from 120 (resp. 85) in the conventional PS approach to 46 (resp. 38) for 6-word (resp. 5-word) multiplication [21]. As a result, KA with threshold 6 (security levels 1 and 3) and threshold 5 (security level 5) consumes 91,449, 288,214, and 783,935 cycles, providing performance improvements of 38.3%, 37.3%, and 25.6% over KA with the optimal threshold shown in Fig. 3.

TABLE IV
RESOURCE UTILIZATION ON ARTIX-7 FPGA (ISE M : UNIFIED CARRYLESS MULTIPLIER, ISE K : KECCAK ACCELERATOR.)

Components	LUTs	Regs	DSPs	BRAM
Ibex [†]	7,014	6,200	10	32
Ibex [†] + M	7,700 (+9.8%)	6,326 (+2.1%)	10	32
Ibex [†] + M + K	11,564 (+64.9%)	8,083 (+30.4%)	10	32
Ibex [‡]	12,780 (+82.2%)	6,882 (+11%)	10	32

[†]:maxperf, [‡]: maxperf-pmp-bmbalanced

IV. EXPERIMENTAL RESULTS

Table IV summarizes the area overhead of the proposed accelerators synthesized using Xilinx Vivado v2022.02 with xc7a35tcg324-1. The carryless multiplier adds 686 LUTs (Look-Up Tables) and 126 FFs (Flip-Flops), increasing LUT and FF utilization by 9.8% and 2.1% over the baseline, respectively. Integrating both the carryless multiplier and the Keccak accelerator increases the overhead to 4,550 LUTs and 1,757 FFs, corresponding to 64.9% and 30.4% increases. The higher cost is mainly due to the complexity of the Keccak permutation and its 1600-bit internal state. Ibex core synthesized with maxperf-pmp-bmbalanced option (denoted as $Ibex^{\ddagger}$) provides several bit-manipulation instructions [6] including native carryless multiplication instructions such as *clmul*, *clmulh*, and *clmulr*⁴ with the increase of the hardware resources compared to the maxperf which is our baseline hardware option.

TABLE V
EXECUTION TIME (KCYCLES) AND PERFORMANCE GAIN (THE SPEEDUP OF THIS WORKS IS MEASURED RELATIVE TO THE HQC BASELINE. ISE M : UNIFIED CARRYLESS MULTIPLIER, ISE K : KECCAK ACCELERATOR)

Version	Level	KeyGen	Encaps	Decaps
HQC Baseline (SW: KA) [1]	HQC-128	47,761.4	95,293.3	144,408.6
	HQC-192	144,247.3	288,153.0	434,200.3
	HQC-256	349,089.3	697,532.4	1,050,061.7
This work (SW: $TC+KA$)	HQC-128	19,412.5 [$\times 2.46$]	38,600.4 [$\times 2.47$]	59,367.0 [$\times 2.43$]
	HQC-192	59,210.4 [$\times 2.44$]	118,079.2 [$\times 2.44$]	179,089.6 [$\times 2.42$]
	HQC-256	136,028.9 [$\times 2.57$]	271,412.0 [$\times 2.57$]	410,800.7 [$\times 2.56$]
This work (SW: KA , ISE: native M)	HQC-128	2,397.3 [$\times 19.92$]	4,542.3 [$\times 20.98$]	8,074.7 [$\times 17.88$]
	HQC-192	6,390.6 [$\times 22.57$]	12,370.1 [$\times 23.29$]	20,161.4 [$\times 21.54$]
	HQC-256	13,639.2 [$\times 25.59$]	26,407.4 [$\times 26.41$]	42,608.1 [$\times 24.64$]
This work (SW: KA , ISE: M)	HQC-128	2,285.2 [$\times 20.90$]	4,291.8 [$\times 22.20$]	7,715.9 [$\times 18.72$]
	HQC-192	6,042.1 [$\times 23.87$]	11,617.5 [$\times 24.80$]	19,089.8 [$\times 22.75$]
	HQC-256	12,491.8 [$\times 27.95$]	24,044.6 [$\times 29.01$]	39,068.1 [$\times 26.88$]
This work (SW: KA , ISE: M , K)	HQC-128	1,679.1 [$\times 28.44$]	3,232.1 [$\times 29.48$]	5,774.2 [$\times 25.01$]
	HQC-192	5,011.2 [$\times 28.78$]	9,604.0 [$\times 30.00$]	15,312.2 [$\times 28.36$]
	HQC-256	10,925.3 [$\times 31.95$]	20,940.5 [$\times 33.31$]	33,191.8 [$\times 31.64$]
This work (SW: $TC+KA$, ISE: M)	HQC-128	1,782.4 [$\times 26.80$]	3,278.9 [$\times 29.06$]	5,785.1 [$\times 24.96$]
	HQC-192	4,455.6 [$\times 32.37$]	8,402.6 [$\times 34.29$]	13,776.9 [$\times 31.52$]
	HQC-256	8,832.5 [$\times 39.52$]	16,655.3 [$\times 41.88$]	26,772.2 [$\times 39.22$]
This work (SW: $TC+KA$, ISE: M , K)	HQC-128	1,144.2 [$\times 40.58$]	2,153.5 [$\times 42.94$]	3,746.5 [$\times 37.56$]
	HQC-192	3,309.5 [$\times 42.33$]	6,192.1 [$\times 45.10$]	9,687.1 [$\times 43.50$]
	HQC-256	6,979.5 [$\times 48.21$]	13,028.7 [$\times 51.47$]	20,086.9 [$\times 50.31$]

Table V summarizes the speedups of our HW/SW co-optimized HQC over the baseline HQC software (all code compiled with the 32-bit RISC-V GCC toolchain at $-O3$). On the SW side, we enhance PM by incorporating a hybrid $TC+KA$ scheme, whereas the HQC reference uses KA only; this SW using $TC+KA$ improves the performance by at least

⁴*clmul* (resp. *clmulh*) returns the lower (higher) 32-bit of 32-bit by 32-bit carryless multiplication.

TABLE VI
RESOURCES, *speedup/eSlice* COMPARISON FOR HQC-128

Version	LUT/FF/BRAM	eSlice	speedup [$\times$] (K/E/D) [<i>speedup/eSlice</i>]
[12] (SW: KA, ISE: M)	3464/3168/0	866	16.95 / 16.37 / 15.70 [19.57 / 18.90 / 18.13]
[12] (SW: KA, ISE: M, K)	8894/3710/0	2224	21.24 / 21.15 / 18.80 [9.55 / 9.51 / 8.45]
[16] (SW: KA, ISE: M, K)	1535/466/0	383.75	23.60 / 24.5 / 21.70 [61.50 / 63.84 / 56.55]
This work (SW: KA, ISE: native M)	5766/682/0	1441.5	19.92 / 20.98 / 17.88 [13.82 / 14.55 / 12.41]
This work (SW: KA, ISE: M)	686/126/0	171.5	20.90 / 22.20 / 18.72 [121.87 / 129.45 / 109.15]
This work (SW: KA, ISE: M, K)	4550/1757/0	1137.5	28.44 / 29.48 / 25.01 [25.00 / 25.92 / 21.99]
This work (SW: TC+KA, ISE: M)	686/126/0	171.5	10.89 / 11.77 / 10.26 [63.50 / 68.63 / 59.83]
This work (SW: TC+KA, ISE: M, K)	4550/1757/0	1137.5	16.49 / 17.39 / 15.44 [14.50 / 15.29 / 13.57]

$\times 2.4$ over the baseline. On the *HW* side, we evaluate two accelerator configurations: (i) the proposed carryless multiplier only and (ii) both the carryless multiplier and the Keccak accelerator. For *PM*, we consider two algorithmic variants: *KA*-only and *TC+KA*. With both accelerators, the *KA*-only implementation achieves speedups of $\times 28.44$ – $\times 31.95$, $\times 29.48$ – $\times 33.31$, and $\times 25.01$ – $\times 31.64$ for KeyGen, Encaps, and Decaps, respectively. The best results are obtained by combining *TC+KA* with both accelerators, reaching $\times 40.58$ – $\times 48.21$, $\times 42.94$ – $\times 51.47$, and $\times 37.56$ – $\times 50.31$ speedups. Moreover, this substantially narrows the performance gap between HQC and ML-KEM (Table I) from $\times 47/ \times 84.5/ \times 108.96$ to $\times 1.13/ \times 1.91/ \times 2.83$ for KeyGen/Encaps/Decaps, respectively, making HQC practical for real embedded deployments from a crypto-agility perspective. The HQC software using the `maxperf-pmp-bmbalanced` configuration can substantially improve performance over the baseline; however, we confirm that the HQC software leveraging the proposed unified multiplier, which is the core contribution of this paper, still achieves higher performance.

Table VI compares our HQC implementations against the latest HQC designs with tightly coupled accelerators [12], [16]. Because a direct comparison is challenging due to differences in the target core, synthesis toolchain, and the polynomial multiplication strategy, we report *speedup/eSlices*—the achieved speedup normalized by the hardware cost in eSlices⁵—to provide a fairer comparison. Since [12] and [16] uses *KA*-based *PM*, we evaluate two software variants of our implementation: *KA*-only and a *TC+KA* hybrid. To enable a clear comparison of hardware efficiency, the speedups reported in Table VI are measured against HQC SW using the same *PM* method. Namely, speedups for the last two rows in Table VI are computed based on the performance of HQC SW with *TC+KA* presented in the third row in Table V as a baseline. This is why the speedups between Table V and Table VI are different when *TC+KA* is used. For the same accelerator configuration, our design consistently achieves higher speedup/eSlices than [12] and [16]. This demonstrates the efficiency of the proposed unified carryless multiplier.

⁵eSlices is computed as $\max\{\frac{LUT}{4} + BRAM \times 128, \frac{FF}{8}\}$ from [22]

REFERENCES

- [1] N. A. e. a. Philippe Gaborit, Carlos Aguilar-Melchor, “Hamming quasi-cyclic (hqc).” <https://pqc-hqc.org/>.
- [2] N. I. of Standards and Technology, “Nist ir 8545 : Status report on the fourth round of the nist post-quantum cryptography standardization process,” tech. rep., NIST, 2025.
- [3] J. Xie, W. Zhao, H. Lee, D. B. Roy, and X. Zhang, “Hardware circuits and systems design for post-quantum cryptography - A tutorial brief,” *IEEE Trans. Circuits Syst. II Express Briefs*, vol. 71, no. 3, pp. 1670–1676, 2024.
- [4] National Institute of Standards and Technology (NIST), “FIPS 203: Module-lattice-based key-encapsulation mechanism standard,” Tech. Rep. 203, NIST, 2024.
- [5] D. C. D. M. A. R. M. S. W. N. R. H. Elaine Barker, Lily Chen and S. Turner, “Considerations for achieving cryptographic agility: Strategies and practices design for post-quantum cryptography - A tutorial brief,” tech. rep., NIST, 2025.
- [6] RISC-V International, “B Extension for Bit Manipulation.” <https://docs.riscv.org/reference/isa/unpriv/b-st-ext.html>.
- [7] T. Fritzmann, G. Sigl, and J. Sepúlveda, “RISQ-V: tightly coupled RISC-V accelerators for post-quantum cryptography,” *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2020, no. 4, pp. 239–280, 2020.
- [8] P. Nannipieri, S. D. Matteo, L. Zulberti, F. Albicocci, S. Saponara, and L. Fanucci, “A RISC-V post quantum cryptography instruction set extension for number theoretic transform to speed-up CRYSTALS algorithms,” *IEEE Access*, vol. 9, pp. 150798–150808, 2021.
- [9] K. Miteloudi, J. W. Bos, O. Bronchain, B. Fay, and J. Renes, “PQ.V.ALUE: post-quantum RISC-V custom ALU extensions on dilithium and kyber,” *IACR Cryptol. ePrint Arch.*, p. 1505, 2023.
- [10] Z. Ye, R. Song, H. Zhang, D. Chen, R. C. Cheung, and K. Huang, “A highly-efficient lattice-based post-quantum cryptography processor for iot applications,” *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2024, no. 2, pp. 130–153, 2024.
- [11] J. Zhang, J. Lu, A. Li, M. Wang, X. Li, T. Huang, L. Chen, and D. Liu, “Super-k: A superscalar CRYSTALS-KYBER processor based on efficient arithmetic array,” *IEEE Trans. Circuits Syst. II Express Briefs*, vol. 71, no. 9, pp. 4286–4290, 2024.
- [12] A. Dolmeta, S. D. Matteo, E. Valea, M. Carmona, A. Loiseau, M. Martina, and G. Masera, “TYRCA: A RISC-V tightly-coupled accelerator for code-based cryptography,” in *Design, Automation & Test in Europe Conference, DATE 2025*, pp. 1–7, 2025.
- [13] National Institute of Standards and Technology (NIST), “FIPS 204: Module-lattice-based digital signature standard,” Tech. Rep. 204, 2024.
- [14] OpenHW Group, “Core-V eXtension interface (CV-X-IF).” <https://github.com/openhwgroup/core-v-xif>.
- [15] OpenHW Group, “CORE-V CV32E40PX RISC-V IP.” <https://github.com/x-heep/cv32e40px>.
- [16] A. Dolmeta, V. Piscopo, G. Masera, M. Martina, and M. Hutter, “HORCRUX - a lightweight PQC-RISC-v eXtension architecture.” *Cryptology ePrint Archive*, Paper 2025/1934, 2025.
- [17] HQC Team, “Hamming Quasi-Cyclic (HQC) Official Implementation, Released on 19th August, 2025.” <https://gitlab.com/pqc-hqc/hqc>.
- [18] National Institute of Standards and Technology (NIST), “FIPS 202: Sha-3 standard: Permutation-based hash and extendable-output functions,” Tech. Rep. 202, NIST, 2015.
- [19] J. M. B. Mera, A. Karmakar, and I. Verbauwhede, “Time-memory trade-off in toom-cook multiplication: an application to module-lattice based cryptography,” *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2020, no. 2, pp. 222–244, 2020.
- [20] lowRISC CIC, “Ibex: 32-bit RISC-V Core.” <https://github.com/lowRISC/ibex>.
- [21] M. Hutter and E. Wenger, “Fast multi-precision multiplication for public-key cryptography on embedded microprocessors,” *J. Cryptol.*, vol. 33, no. 4, pp. 1442–1460, 2020.
- [22] F. Antognazza, A. Barengi, G. Pelosi, and R. Susella, “A high efficiency hardware design for the post-quantum KEM HQC,” in *IEEE International Symposium on Hardware Oriented Security and Trust, HOST 2024, Tysons Corner, VA, USA, May 6-9, 2024*, pp. 431–441, IEEE, 2024.